



Live Speech Translation Application: Infrastructure & Cost Planning Document

Open-Source Model Deployment | Server Configuration | Cost Analysis

Bidirectional Real-Time Translation (English-French) | 1000 Concurrent Users

Prepared by: Technical Architecture Team

Date: May 29, 2026

Classification: Internal / Confidential

Table of Contents

1. Executive Summary.....	1
2. Application Overview & Architecture.....	1
2.1 Application Description.....	1
2.2 Core Technical Requirements	2
2.3 Processing Pipeline Architecture.....	2
3. Open-Source Model Selection.....	2
3.1 Automatic Speech Recognition (ASR).....	2
3.2 Machine Translation (MT)	3
3.3 Text-to-Speech (TTS).....	4
3.4 Pipeline Latency Budget.....	5
4. Server Configuration: Production (1,000 Concurrent Users).....	6
4.1 Architecture Design.....	6
4.2 GPU Server Requirements	6
4.3 CPU/API Server Requirements	7
4.4 Load Balancing & Networking	7
4.5 Storage Requirements.....	8
5. Server Configuration: Testing & Development.....	8
5.1 Testing Environment	8
5.2 Development Workstation.....	9
6. Data Requirements for Model Fine-Tuning.....	9
6.1 ASR Training Data	10
6.2 Translation Training Data.....	10
6.3 TTS Training Data.....	10
6.4 Data Sources & Licensing Summary	11
7. Cost Breakdown.....	11
7.1 Testing Phase Costs.....	11
7.2 Development Phase Costs	12
7.3 Production Infrastructure Costs (Monthly)	13
7.4 Two-Day Sprint Deployment Cost.....	14

7.5 Total Cost Summary	14
8. Open-Source vs API-Based Cost Comparison	15
9. Latency Optimization Strategies	16
9.1 Streaming ASR with Partial Results	16
9.2 Model Quantization and Optimization	16
9.3 GPU Batching and Concurrent Inference	16
9.4 Audio Compression and Network Optimization	17
10. Risk Analysis & Mitigation	17
11. Recommendations & Next Steps	18

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary

This document presents a comprehensive infrastructure and cost planning analysis for a live speech translation application that delivers bidirectional, real-time translation between English and French. The application is designed to receive speech input from User A in English, translate it in real time, and deliver the translated speech output to User B in French, and vice versa. The primary technical requirement is achieving very low end-to-end latency, ensuring that users experience natural, conversational interaction without perceptible delays.

The solution is built entirely on open-source models rather than commercial APIs, providing full control over data privacy, customization, and long-term cost predictability. The core processing pipeline consists of three stages: Automatic Speech Recognition (ASR) to convert speech to text, Machine Translation (MT) to translate text between languages, and Text-to-Speech (TTS) to synthesize translated speech output. Each stage has been carefully evaluated against latency, accuracy, and deployment feasibility criteria.

This document defines the server configurations required for both a production deployment supporting 1,000 concurrent users and a smaller-scale testing and development environment. It also details the data requirements for model fine-tuning, provides a full cost breakdown across testing, development, and production phases, and includes a comparative analysis of open-source versus API-based approaches. The total estimated monthly infrastructure cost for production with 1,000 concurrent users is approximately \$8,700 to \$11,200, while the initial development and testing investment over a two-month period is estimated at \$14,500 to \$21,000.

2. Application Overview & Architecture

2.1 Application Description

The live speech translation application enables two users speaking different languages to communicate in real time through a seamless bidirectional translation pipeline. When User A speaks in English, the system captures the audio stream, transcribes it into English text using an ASR model, translates the text into French using an MT model, and synthesizes the French translation as spoken audio using a TTS model. This output is then streamed to User B. Simultaneously, the reverse pipeline handles User B's French speech, translating it back to English for User A. The entire round-trip process must complete within a latency budget of under 500 milliseconds to maintain natural conversational flow.

The application targets a production environment that must support 1,000 concurrent user pairs (2,000 individual audio streams), demanding significant GPU compute resources for

real-time inference. The architecture must be horizontally scalable, fault-tolerant, and capable of handling variable load patterns typical of communication applications, including peak usage surges during business hours.

2.2 Core Technical Requirements

The system must satisfy several critical technical requirements that directly influence infrastructure sizing and model selection. First, end-to-end latency from speech capture to translated speech output must remain below 500 milliseconds under normal operating conditions, with a target of 300 milliseconds for optimal user experience. Second, the system must support bidirectional translation simultaneously for each user pair, effectively doubling the per-session compute requirement. Third, all models must be open-source and self-hosted to ensure data sovereignty, eliminate per-request API costs, and allow fine-tuning on domain-specific data. Fourth, the architecture must scale horizontally to handle 1,000 concurrent sessions with graceful degradation under peak load. Fifth, audio quality must be maintained at a minimum of 16 kHz sampling rate with minimal artifact introduction during the ASR-TTS round trip.

2.3 Processing Pipeline Architecture

The processing pipeline for each translation direction consists of three sequential model inference stages, each contributing to the overall latency budget. The ASR stage receives a streaming audio input, processes it in chunked windows (typically 200-500 ms chunks), and produces text transcriptions. The MT stage takes the transcribed text and performs translation, which can be triggered on partial transcriptions for reduced latency. The TTS stage synthesizes the translated text into speech audio, which is streamed back to the receiving user. A WebSocket-based signaling layer coordinates audio streaming between users and the processing pipeline, while a load balancer distributes inference requests across GPU server clusters.

3. Open-Source Model Selection

Model selection is the most critical decision affecting both latency and infrastructure cost. Each model in the pipeline has been evaluated against four criteria: inference latency on GPU, accuracy (Word Error Rate for ASR, BLEU score for MT, Mean Opinion Score for TTS), model size and memory footprint, and community support and maintenance activity. The following sections present the recommended models with justification.

3.1 Automatic Speech Recognition (ASR)

The primary recommendation for ASR is Whisper Large V3, developed by OpenAI and released under the MIT license. Whisper Large V3 achieves state-of-the-art accuracy with a Word Error Rate (WER) of approximately 5.1% on the LibriSpeech benchmark for English and competitive performance on French. The model has 1.55 billion parameters and requires approximately 3.1 GB of GPU memory in FP16 precision. On a single NVIDIA A10G GPU, Whisper Large V3 processes audio at roughly 15-20x real-time speed for batch size 1, meaning a 10-second audio clip completes transcription in approximately 500-700 milliseconds of compute time.

For scenarios where lower latency is prioritized over maximum accuracy, Whisper Medium (769 million parameters) or Whisper Small (244 million parameters) provide compelling alternatives. Whisper Medium runs approximately 2.5x faster than Large V3 with a modest WER increase of 1-2 percentage points, while Whisper Small offers approximately 5x faster inference at the cost of a 3-4 point WER degradation. For production deployment with 1,000 users, a tiered approach is recommended: use Whisper Medium for initial low-latency partial transcriptions and Whisper Large V3 for final accurate transcriptions.

Table 1: ASR Model Comparison

Model	Parameters	GPU Memory (FP16)	Inference Speed	WER (English)	License
Whisper Large V3	1.55B	3.1 GB	15-20x RT	5.1%	MIT
Whisper Medium	769M	1.5 GB	35-45x RT	6.8%	MIT
Whisper Small	244M	0.5 GB	70-90x RT	8.5%	MIT
Faster-Whisper Large V3	1.55B (CT2)	3.1 GB	25-35x RT	5.1%	MIT

3.2 Machine Translation (MT)

For machine translation, the recommended model is NLLB-200 (No Language Left Behind), developed by Meta and released under the CC-BY-NC 4.0 license. NLLB-200 Distilled 600M provides an excellent balance between translation quality and inference speed, achieving a BLEU score of approximately 38-42 for English-to-French and 36-40 for French-to-English on standard benchmarks. The distilled 600M parameter model requires only 1.2 GB of GPU memory in FP16 and achieves inference latency of approximately 30-50 milliseconds per sentence on an NVIDIA A10G GPU.

For deployments requiring higher translation quality and where a non-commercial license is acceptable, the NLLB-200 1.3B Distilled variant provides a measurable quality improvement of 2-3 BLEU points with approximately 2.6 GB GPU memory and 50-80 ms

inference latency. If a fully permissive commercial license is required, the M2M100 418M model by Facebook provides a viable alternative with Apache 2.0 licensing, though with slightly lower BLEU scores of approximately 32-36 for the English-French pair. The Opus-MT models by Helsinki NLP offer another Apache 2.0 licensed option with competitive quality for specific language pairs but lack the flexibility of multilingual models.

Table 2: Machine Translation Model Comparison

Model	Parameters	GPU Memory (FP16)	Latency/Sentence	BLEU (EN-FR)	License
NLLB-200 Distilled 600M	600M	1.2 GB	30-50 ms	38-42	CC-BY-NC 4.0
NLLB-200 Distilled 1.3B	1.3B	2.6 GB	50-80 ms	41-45	CC-BY-NC 4.0
M2M100 418M	418M	0.8 GB	25-40 ms	32-36	Apache 2.0
Opus-MT EN-FR	~300M	0.6 GB	20-35 ms	34-38	Apache 2.0

3.3 Text-to-Speech (TTS)

The recommended TTS model is Coqui TTS (XTTS V2), which provides high-quality multilingual speech synthesis with voice cloning capabilities. XTTS V2 supports English and French natively, produces natural-sounding speech with a Mean Opinion Score (MOS) of approximately 4.0-4.2, and can synthesize speech in approximately 100-200 milliseconds for a typical sentence on an NVIDIA A10G GPU. The model requires approximately 1.8 GB of GPU memory in FP16. The license is MPL 2.0 for the codebase with model weights available under a non-commercial research license, though commercial licensing options are available through Coqui.

For a fully open-source commercial alternative, VITS (Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech) offers Apache 2.0 licensed synthesis with competitive quality. VITS models are typically smaller (50-100M parameters) and faster (50-100 ms per sentence) but require separate models for each language and lack the voice cloning feature. The combination of a French VITS model and an English VITS model would serve the bidirectional use case effectively while maintaining full commercial licensing freedom.

Table 3: TTS Model Comparison

Model	Parameters	GPU Memory (FP16)	Latency/Sentence	MOS	License
Coqui XTTS V2	~1.8B	1.8 GB	100-200 ms	4.0-4.2	MPL 2.0 / NC
VITS (EN)	~100M	0.4 GB	50-100 ms	3.8-4.0	Apache 2.0
VITS (FR)	~100M	0.4 GB	50-100 ms	3.7-3.9	Apache 2.0
Piper TTS	~20M	0.1 GB	10-30 ms	3.5-3.8	MIT

3.4 Pipeline Latency Budget

The end-to-end latency budget must be carefully managed across all three pipeline stages plus network overhead. Based on the recommended model selections and assuming streaming ASR with partial transcription, the target latency breakdown is as follows. ASR processing adds approximately 100-150 milliseconds when using Whisper Medium for partial results, with an additional 100-150 ms for final results from Whisper Large V3. Machine translation contributes approximately 30-50 milliseconds per sentence with NLLB-200 Distilled 600M. TTS synthesis adds approximately 100-200 milliseconds per sentence with XTTS V2. Network and buffering overhead accounts for approximately 30-50 milliseconds for WebSocket audio streaming and chunk buffering.

Under optimal conditions, the total end-to-end latency for partial results is approximately 260-400 milliseconds, which falls well within the 500 ms target. For final complete transcriptions, the latency may reach 360-500 ms. When using the faster VITS TTS model and M2M100 translation, the latency can be reduced to approximately 200-300 ms for partial results, providing an even more responsive user experience at the cost of slightly lower translation quality and speech naturalness.

Table 4: End-to-End Latency Budget Breakdown

Pipeline Stage	Model	Optimistic (ms)	Typical (ms)	Pessimistic (ms)
ASR (Partial)	Whisper Medium	80	120	150
ASR (Final)	Whisper Large V3	150	250	350
Translation	NLLB-200 600M	30	40	50
TTS	Coqui XTTS V2	100	150	200
Network/Buffer	WebSocket Streaming	30	40	50

Pipeline Stage	Model	Optimistic (ms)	Typical (ms)	Pessimistic (ms)
Total (Partial)	All stages	240	350	450
Total (Final)	All stages	360	480	650

4. Server Configuration: Production (1,000 Concurrent Users)

4.1 Architecture Design

The production architecture is designed around a microservices approach with dedicated GPU server clusters for each model type, enabling independent scaling and fault isolation. Each translation session requires two parallel inference pipelines (one per direction), meaning 1,000 concurrent user pairs translate to 2,000 simultaneous inference streams. The architecture consists of three GPU inference clusters (ASR, MT, TTS), a CPU-based API gateway and signaling layer, a load balancer for traffic distribution, and object storage for model artifacts and audio caching.

The ASR cluster is the most GPU-intensive component, as each Whisper model instance can handle approximately 8-12 concurrent streams on an NVIDIA A10G GPU with batching. To support 2,000 concurrent ASR streams, the cluster requires approximately 20-25 GPU instances. The MT cluster is relatively lightweight due to the small model size and fast inference, with each GPU handling approximately 50-80 concurrent translation requests. The TTS cluster sits between the two, with each GPU handling approximately 15-25 concurrent synthesis streams depending on sentence length and audio quality settings.

4.2 GPU Server Requirements

The primary GPU server configuration recommended for production is the NVIDIA A10G (24 GB VRAM) which provides an optimal balance of compute performance, memory capacity, and cost. Each A10G GPU can comfortably host one Whisper Large V3 instance or multiple smaller models (2-3 Whisper Medium instances or 4-6 NLLB-200 instances). For organizations requiring higher single-GPU performance, the NVIDIA A100 (40/80 GB) can host multiple model instances on a single GPU, reducing server count and network overhead but at a significantly higher per-unit cost.

Table 5: Production GPU Server Specifications

Component	ASR Cluster	MT Cluster	TTS Cluster
GPU Model	NVIDIA A10G (24 GB)	NVIDIA A10G (24 GB)	NVIDIA A10G (24 GB)

Component	ASR Cluster	MT Cluster	TTS Cluster
GPUs per Server	4	2	2
Concurrent Streams/GPU	8-12	50-80	15-25
Number of Servers	6-7	2-3	4-5
Total GPUs	24-28	4-6	8-10
CPU (per server)	64 vCPU, AMD EPYC	32 vCPU, AMD EPYC	32 vCPU, AMD EPYC
RAM (per server)	128 GB DDR4	64 GB DDR4	64 GB DDR4
Storage (per server)	500 GB NVMe SSD	200 GB NVMe SSD	500 GB NVMe SSD
Network	10 Gbps	10 Gbps	10 Gbps

4.3 CPU/API Server Requirements

The CPU-based infrastructure handles API gateway routing, WebSocket connection management, audio preprocessing and postprocessing, session state management, authentication and authorization, logging and monitoring, and model artifact distribution. These services are stateless where possible and deployed behind a load balancer for high availability. The API gateway must handle approximately 2,000 concurrent WebSocket connections, each carrying bidirectional audio streams at approximately 256 kbps (16 kHz, 16-bit audio, compressed with Opus codec).

Table 6: CPU/API Server Specifications

Server Type	Count	vCPU	RAM	Storage	Purpose
API Gateway	3	16	32 GB	100 GB SSD	WebSocket mgmt, routing
Audio Processing	3	16	32 GB	100 GB SSD	Pre/post processing
Session Manager	2	8	16 GB	50 GB SSD	State, auth, logging
Monitoring Stack	2	8	32 GB	500 GB SSD	Prometheus, Grafana, ELK
Model Registry	1	8	16 GB	1 TB SSD	Model artifact storage

4.4 Load Balancing & Networking

The load balancing layer is critical for distributing inference traffic evenly across GPU clusters and ensuring session affinity where required. The recommended configuration uses an Application Load Balancer (ALB) for WebSocket traffic with sticky sessions to maintain audio stream continuity, and a Network Load Balancer (NLB) for internal service-to-service communication between microservices. Each GPU cluster has its own internal load balancer that routes inference requests to the least-loaded GPU instance.

Network bandwidth requirements are significant for a 1,000-user deployment. Each user pair generates approximately 512 kbps of audio traffic (256 kbps per direction), totaling approximately 500 Mbps of sustained audio bandwidth for 1,000 pairs. Internal network traffic between the API gateway, GPU clusters, and audio processing services adds approximately 200-300 Mbps, bringing the total estimated network bandwidth to approximately 700-800 Mbps. A 10 Gbps internal network provides ample headroom for peak traffic and future scaling.

4.5 Storage Requirements

Storage is required for model artifacts, audio caching, session logs, and application code. Model artifacts for the full pipeline (Whisper Large V3 at 3.1 GB, NLLB-200 Distilled 600M at 1.2 GB, XTTS V2 at 1.8 GB, plus Whisper Medium at 1.5 GB) total approximately 7.6 GB per deployment replica. With 6-7 ASR servers, 2-3 MT servers, and 4-5 TTS servers, total model storage across the cluster is approximately 60-80 GB. Audio caching for active sessions requires approximately 2-4 GB per 1,000 sessions. Session logs and monitoring data generate approximately 50-100 GB per month depending on retention policy.

5. Server Configuration: Testing & Development

5.1 Testing Environment

The testing environment should mirror the production architecture at a reduced scale, supporting approximately 20-50 concurrent user pairs for functional testing, integration testing, and performance benchmarking. This environment does not need to handle production load but must be architecturally identical to validate model deployment, pipeline integration, and latency characteristics under controlled conditions. A single multi-GPU server with 2-4 A10G or equivalent GPUs can serve all three model types simultaneously for testing purposes, with each GPU hosting one model instance.

Table 7: Testing Environment Server Configuration

Component	Specification	Purpose
GPU Server	1x with 4x A10G (24 GB), 64 vCPU, 128 GB RAM, 1 TB NVMe	Host ASR + MT + TTS models
API Server	1x 16 vCPU, 32 GB RAM, 200 GB SSD	Gateway, audio processing, sessions
Monitoring	1x 8 vCPU, 16 GB RAM, 200 GB SSD	Logging, metrics, dashboards
Load Balancer	Cloud-managed ALB	Traffic distribution (testing)

5.2 Development Workstation

For individual developer workstations, a single NVIDIA RTX 4090 (24 GB VRAM) or RTX 3090 (24 GB VRAM) provides sufficient GPU memory to run the complete pipeline locally with one model of each type loaded simultaneously. This enables rapid iteration during development and debugging without incurring cloud GPU costs. For developers focused on individual model components, even an NVIDIA RTX 4070 (12 GB VRAM) or RTX 3080 (10 GB VRAM) is sufficient to run a single model with quantized weights.

Table 8: Development Workstation Configuration

Component	Full Pipeline Spec	Single Model Spec
GPU	NVIDIA RTX 4090 (24 GB)	NVIDIA RTX 4070 (12 GB)
CPU	AMD Ryzen 9 / Intel i9 (16+ cores)	AMD Ryzen 7 / Intel i7 (8+ cores)
RAM	64 GB DDR5	32 GB DDR5
Storage	2 TB NVMe SSD	1 TB NVMe SSD
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Estimated Cost	\$3,500 - \$4,500	\$1,500 - \$2,200

6. Data Requirements for Model Fine-Tuning

While the selected open-source models provide strong baseline performance out of the box, fine-tuning on domain-specific data significantly improves accuracy and user experience. The data requirements vary by model type and target use case. For a conversational speech translation application, the most impactful fine-tuning targets are conversational ASR (handling disfluencies, overlapping speech, and domain vocabulary), translation of conversational and informal language, and TTS voice consistency and naturalness for the specific output language.

6.1 ASR Training Data

Fine-tuning the ASR model requires paired audio and transcript data that reflects the target domain. For conversational speech, the recommended datasets include the Mozilla Common Voice corpus (over 20,000 hours across 100+ languages, CC0 license), the LibriSpeech dataset (1,000 hours of read English speech, CC-BY 4.0), the Multilingual LibriSpeech corpus (50,000 hours across 8 languages), and the Gigaspeech dataset (10,000 hours of English audiobook data). For French-specific ASR, the CSS10 French dataset and the African-accented French datasets from Mozilla Common Voice provide valuable supplementary training data.

The minimum viable fine-tuning dataset for noticeable improvement is approximately 100-200 hours of domain-matched audio per language. For optimal results, 500-1,000 hours per language is recommended. Data preparation requires audio normalization (16 kHz, mono, 16-bit), transcript alignment and cleaning, speaker segmentation for multi-speaker audio, and train/validation/test splitting (80/10/10). The estimated storage requirement for 1,000 hours of 16 kHz audio is approximately 115 GB, with an additional 50-100 GB for intermediate processing artifacts and feature caches.

6.2 Translation Training Data

Fine-tuning the translation model requires parallel corpora of English-French sentence pairs that reflect conversational, informal language patterns. Key datasets include the OPUS parallel corpus (over 60 million English-French sentence pairs from diverse sources), the WMT translation competition datasets (high-quality curated parallel text), the Tatoeba project (over 1 million community-contributed sentence pairs), and the EuroParl corpus (approximately 2 million sentence pairs from European Parliament proceedings). For conversational quality, the OpenSubtitles dataset within OPUS provides movie and TV subtitle translations that better reflect informal speech patterns.

A minimum of 500,000 sentence pairs is recommended for fine-tuning, with 2-5 million pairs providing significant quality gains. Data cleaning is essential and includes removing misaligned pairs, filtering by length ratio (targeting 1:1.3 for EN-FR), deduplication, and language identification verification. The storage requirement for 5 million sentence pairs with metadata is approximately 2-5 GB, which is modest compared to ASR data.

6.3 TTS Training Data

TTS fine-tuning requires high-quality audio recordings with precise text alignment. For voice customization, 2-10 hours of clean, single-speaker audio at 22 kHz or higher sampling rate is typically sufficient. For general quality improvement, datasets such as the VCTK corpus (44 hours of English speech from 109 speakers), the LibriTTS corpus (585 hours at

24 kHz), and the French Multilingual LibriSpeech (MLS) dataset provide substantial training data. The Coqui TTS framework also provides tools for generating synthetic training data and aligning existing audio with transcripts.

If voice cloning is desired (maintaining User A's voice characteristics in the translated output for User B), approximately 3-6 seconds of reference audio per speaker is sufficient with XTTS V2. This is a powerful feature for maintaining speaker identity in translated speech but raises ethical considerations around voice impersonation that should be addressed through consent mechanisms and disclaimers.

6.4 Data Sources & Licensing Summary

Table 9: Training Data Sources and Licensing

Dataset	Type	Size	Languages	License
Mozilla Common Voice	ASR	20,000+ hours	100+ languages	CC0
LibriSpeech	ASR	1,000 hours	English	CC-BY 4.0
Multilingual LibriSpeech	ASR	50,000 hours	8 languages	CC-BY 4.0
OPUS Parallel Corpus	MT	60M+ sentence pairs	EN-FR among others	Mixed (per subcorpus)
WMT Datasets	MT	Varies by year	EN-FR included	Varies
OpenSubtitles (OPUS)	MT	~40M sentence pairs	EN-FR	CC-BY-SA
VCTK Corpus	TTS	44 hours	English	CC-BY 4.0
LibriTTS	TTS	585 hours	English	CC-BY 4.0

7. Cost Breakdown

This section provides a detailed cost analysis across three phases: testing, development, and production deployment. All costs are based on major cloud provider pricing (AWS, GCP, or Azure) as of mid-2026, using on-demand GPU instances unless otherwise noted. Significant savings (30-60%) are available through reserved instances, spot/preemptible instances, and committed use discounts, which are noted where applicable.

7.1 Testing Phase Costs

The testing phase is estimated to last 2-4 weeks and involves deploying the full pipeline in a reduced-scale environment, running functional and performance tests, validating latency

targets, and conducting model fine-tuning experiments. The primary costs during this phase are GPU compute for model inference and fine-tuning, and cloud infrastructure for the testing environment.

Table 10: Testing Phase Cost Breakdown (2-4 Weeks)

Item	Specification	Quantity	Unit Cost	Duration	Total
GPU Server (4x A10G)	p4d.24xlarge or equiv.	1	\$7.50/hr	4 weeks (672 hrs)	\$5,040
API Server	c5.4xlarge or equiv.	1	\$0.68/hr	4 weeks (672 hrs)	\$457
Monitoring Server	m5.2xlarge	1	\$0.38/hr	4 weeks (672 hrs)	\$255
Load Balancer	Cloud ALB	1	\$0.02/hr + data	4 weeks	\$50
Storage (S3/EBS)	Model + Data artifacts	500 GB	\$0.10/GB/mo	1 month	\$50
Data Transfer	Internal + egress	~2 TB	Variable	4 weeks	\$150
Fine-Tuning Compute	GPU hours for experiments	~200 hrs	\$7.50/hr	Ad hoc	\$1,500
Testing Total					\$7,502

7.2 Development Phase Costs

The development phase spans approximately 6-8 weeks and includes application development, pipeline integration, model optimization (quantization, batching strategies), WebSocket implementation, UI/UX development, and comprehensive testing. Development costs include both cloud infrastructure and developer workstations.

Table 11: Development Phase Cost Breakdown (6-8 Weeks)

Item	Specification	Quantity	Unit Cost	Duration	Total
GPU Server (4x A10G)	For staging/integration	1	\$7.50/hr	8 weeks (1344 hrs)	\$10,080
API Servers	2x for staging	2	\$0.68/hr	8 weeks (1344 hrs)	\$1,826
Monitoring Stack	Staging environment	1	\$0.38/hr	8 weeks (1344 hrs)	\$511
Storage	Models, data, logs	1 TB	\$0.10/GB/mo	2 months	\$200

Item	Specification	Quantity	Unit Cost	Duration	Total
Developer Workstations	RTX 4090 systems	3	\$4,000	One-time	\$12,000
Cloud Dev GPU	Single A10G for ad-hoc	~500 hrs	\$3.50/hr	8 weeks	\$1,750
Data Transfer	Internal + egress	~5 TB	Variable	8 weeks	\$375
Development Total					\$26,742

7.3 Production Infrastructure Costs (Monthly)

The production monthly cost represents the ongoing operational expense for serving 1,000 concurrent users. This is the most significant cost component and is highly sensitive to GPU pricing, instance type selection, and commitment level. The estimates below use on-demand pricing; reserved instance pricing can reduce GPU costs by 30-50% with a 1-year commitment.

Table 12: Production Monthly Cost (1,000 Concurrent Users)

Item	Specification	Quantity	Monthly Unit Cost	Monthly Total
ASR GPU Servers	4x A10G each (g5.12xlarge)	7	\$4,208	\$29,456
MT GPU Servers	2x A10G each (g5.8xlarge)	3	\$2,572	\$7,716
TTS GPU Servers	2x A10G each (g5.8xlarge)	5	\$2,572	\$12,860
API Gateway Servers	c5.4xlarge	3	\$490	\$1,470
Audio Processing Servers	c5.4xlarge	3	\$490	\$1,470
Session Manager Servers	c5.2xlarge	2	\$245	\$490
Monitoring Stack	m5.2xlarge	2	\$274	\$548
Model Registry Server	m5.2xlarge	1	\$274	\$274
Load Balancers	ALB + NLB	2	\$150 (est.)	\$300

Item	Specification	Quantity	Monthly Unit Cost	Monthly Total
Storage (S3 + EBS)	2 TB models + logs	Lot	\$250 (est.)	\$250
Data Transfer	~30 TB/month	Lot	\$0.09/GB	\$2,700
Production Total (On-Demand)				\$57,534
Production Total (Reserved 1yr)		(-40% GPU)		\$34,521

The on-demand production cost of approximately \$57,500 per month reflects worst-case pricing. With 1-year reserved instances (which provide approximately 40% GPU savings), the monthly cost drops to approximately \$34,500. For further optimization, spot/preemptible instances can be used for non-critical batch processing workloads like model fine-tuning and log analysis, potentially reducing those workloads by 60-70%. The effective production cost with reserved instances and selective spot usage is estimated at \$30,000-\$35,000 per month.

7.4 Two-Day Sprint Deployment Cost

For a rapid two-day deployment sprint, the infrastructure cost is limited to the compute resources needed to stand up the production environment, deploy models, run smoke tests, and begin serving traffic. This assumes the development and testing phases are complete and all model artifacts, Docker images, and configuration files are ready for deployment.

Table 13: Two-Day Deployment Sprint Cost

Item	Specification	Quantity	Duration	Cost
Production GPU Servers	15 servers (all clusters)	15	48 hours	\$3,840
Production CPU Servers	11 servers (all types)	11	48 hours	\$320
Load Balancers	ALB + NLB	2	48 hours	\$20
Storage	Model deployment + cache	Lot	2 days	\$20
Data Transfer	Deployment artifacts	~500 GB	One-time	\$45
DevOps Labor	2 engineers x 16 hours	2	2 days	Included in salary
Two-Day Sprint Total				\$4,245

7.5 Total Cost Summary

Table 14: Total Project Cost Summary

Phase	Duration	Cost (Low Estimate)	Cost (High Estimate)
Testing	2-4 weeks	\$6,000	\$9,000
Development	6-8 weeks	\$20,000	\$30,000
Two-Day Deployment Sprint	2 days	\$3,500	\$5,000
Production (1st month)	1 month	\$30,000	\$57,500
Production (Reserved, monthly)	Ongoing	\$30,000	\$35,000
Total Initial Investment	~3 months	\$59,500	\$101,500

8. Open-Source vs API-Based Cost Comparison

A critical question for this project is whether using open-source self-hosted models is more cost-effective than consuming commercial API services. This section provides a detailed comparison using major API providers for equivalent functionality. The comparison considers ASR (using Google Speech-to-Text, AWS Transcribe, or Deepgram), translation (using Google Translation API, AWS Translate, or DeepL), and TTS (using Google Text-to-Speech, AWS Polly, or ElevenLabs).

Table 15: Open-Source vs API Cost Comparison (Monthly, 1000 Users)

Component	Open-Source (Self-Hosted)	API (Per-Request Pricing)	Savings with Open-Source
ASR	\$17,220 (7 servers reserved)	\$14,400 (\$0.024/min x 1K users x 10K min)	Higher initially, lower at scale
Translation	\$4,630 (3 servers reserved)	\$4,500 (\$0.000015/char x 300B chars)	Comparable
TTS	\$7,716 (5 servers reserved)	\$21,600 (\$0.024/min x 1K users x 900 min)	64% savings
Infrastructure Overhead	\$5,532 (CPU + networking)	\$1,000 (minimal infra)	API saves here
Total Monthly	\$35,098	\$41,500	15% savings, growing with scale

The open-source approach becomes increasingly cost-effective as usage scales beyond 1,000 concurrent users, since the per-request cost of APIs grows linearly while self-hosted infrastructure costs grow sub-linearly due to improved GPU utilization at higher batch sizes. At 2,000 concurrent users, the open-source approach saves approximately 25-35% compared

to APIs, and at 5,000 concurrent users, savings exceed 40-50%. Additionally, self-hosting provides data privacy guarantees, eliminates vendor lock-in, enables custom fine-tuning, and offers predictable monthly costs that are not subject to API pricing changes.

9. Latency Optimization Strategies

Achieving sub-500 ms end-to-end latency requires careful optimization across all pipeline stages. This section outlines the key strategies that should be implemented during development to meet the latency targets consistently under production load.

9.1 Streaming ASR with Partial Results

Rather than waiting for the user to complete a full utterance before beginning translation, the ASR model should process audio in streaming mode with partial transcription results emitted every 200-300 milliseconds. These partial transcriptions can be immediately forwarded to the translation model, allowing the system to begin generating translated output before the speaker has finished their sentence. The final, more accurate transcription replaces the partial result once the utterance is complete. This approach reduces perceived latency by 30-50% compared to batch processing. The Faster-Whisper library (using CTranslate2 inference engine) provides native support for streaming transcription with VAD (Voice Activity Detection) integration.

9.2 Model Quantization and Optimization

Model quantization reduces inference latency and GPU memory usage by converting model weights from FP16 (16-bit floating point) to INT8 (8-bit integer) or even INT4 formats. For Whisper models, INT8 quantization typically reduces inference time by 30-40% with less than 1% WER degradation. For NLLB translation models, INT8 quantization provides approximately 25-35% speedup with negligible BLEU score impact. The ONNX Runtime and TensorRT inference engines provide optimized quantization paths for production deployment. Additionally, model distillation (using the already-distilled NLLB-200 models) and knowledge distillation during fine-tuning can further reduce model size while maintaining accuracy.

9.3 GPU Batching and Concurrent Inference

Batching multiple inference requests on the same GPU dramatically improves throughput and per-request latency. For ASR, dynamic batching of audio chunks from multiple concurrent streams can improve GPU utilization from 20-30% (single-stream) to 80-90% (batched). For translation, batch sizes of 8-16 sentences provide optimal throughput with minimal queuing delay. For TTS, batching sentence-level synthesis requests reduces per-

sentence latency by approximately 40-60% at batch size 8. The NVIDIA Triton Inference Server provides production-grade dynamic batching with configurable timeout and batch size parameters.

9.4 Audio Compression and Network Optimization

Audio compression using the Opus codec at 32-64 kbps provides excellent speech quality at approximately one-quarter the bandwidth of uncompressed 16 kHz PCM audio (256 kbps). This reduces network latency, especially for users on mobile connections. WebSocket multiplexing allows a single connection to carry both audio streams (input and translated output) with minimal framing overhead. Edge deployment of the ASR model (running a smaller Whisper Small or Tiny model on the client device for initial transcription) can eliminate the network round-trip for the ASR stage, reducing end-to-end latency by 100-150 milliseconds.

10. Risk Analysis & Mitigation

Table 16: Key Risks and Mitigation Strategies

Risk	Impact	Probability	Mitigation
Latency exceeds 500 ms target	High	Medium	Implement streaming ASR, quantization, and batching optimizations; fall back to smaller models if needed
GPU cost exceeds budget	High	Medium	Use reserved instances, optimize batch sizes, implement auto-scaling, evaluate spot instances for non-critical workloads
Translation quality insufficient	Medium	Low	Fine-tune on domain-specific conversational data; evaluate alternative models (M2M100, Opus-MT)
TTS voice quality unnatural	Medium	Medium	Invest in voice cloning with XTTS V2; evaluate ElevenLabs API as fallback for critical use cases
License compliance issues	High	Low	Audit all model licenses before deployment; use Apache 2.0 alternatives (M2M100, VITS) if commercial license needed
Single point of failure in GPU cluster	High	Low	Deploy across multiple availability zones; implement health checks and automatic failover

Risk	Impact	Probability	Mitigation
Scaling beyond 1000 concurrent users	Medium	Medium	Architecture supports horizontal scaling; add GPU servers incrementally; implement auto-scaling policies

11. Recommendations & Next Steps

Based on the analysis presented in this document, the following recommendations are provided for the successful deployment of the live speech translation application. These recommendations are prioritized by impact and urgency, with the most critical items listed first.

1. Begin with the testing phase immediately. Deploy the full pipeline on a single 4x A10G server and validate end-to-end latency targets before committing to production infrastructure. This de-risks the most significant technical uncertainty and provides empirical data for production sizing. The estimated cost of \$6,000-\$9,000 for the testing phase is a small investment compared to the potential cost of over-provisioned production infrastructure.

2. Use the recommended model stack for initial deployment. Whisper Large V3 (with Faster-Whisper CTranslate2 backend) for ASR, NLLB-200 Distilled 600M for translation, and Coqui XTTS V2 for TTS. This combination provides the best balance of quality and latency. If the NLLB CC-BY-NC license is problematic for commercial use, substitute M2M100 418M or Opus-MT with Apache 2.0 licensing.

3. Implement streaming ASR from day one. Streaming partial results is the single most impactful optimization for perceived latency. Design the pipeline architecture to support partial transcription from the beginning rather than retrofitting it later. Use Faster-Whisper with VAD integration for production-grade streaming.

4. Invest in reserved instances for production. With 1-year reserved instance commitments, GPU costs decrease by approximately 40%, reducing the monthly production cost from \$57,500 to approximately \$34,500. This represents savings of over \$275,000 annually and provides cost predictability for budgeting.

5. Plan for horizontal scaling. Design the GPU cluster architecture with auto-scaling in mind. Use Kubernetes or a similar orchestration platform for model deployment, enabling automatic scale-out during peak hours and scale-in during low-traffic periods. This optimization alone can reduce average monthly costs by 15-25%.

6. Allocate budget for fine-tuning. Budget approximately 200-300 GPU hours (\$1,500-\$2,250) for model fine-tuning experiments during the testing phase. Fine-tuning on

conversational data typically improves WER by 15-25% and BLEU by 3-5 points, directly improving user experience and reducing support costs.